

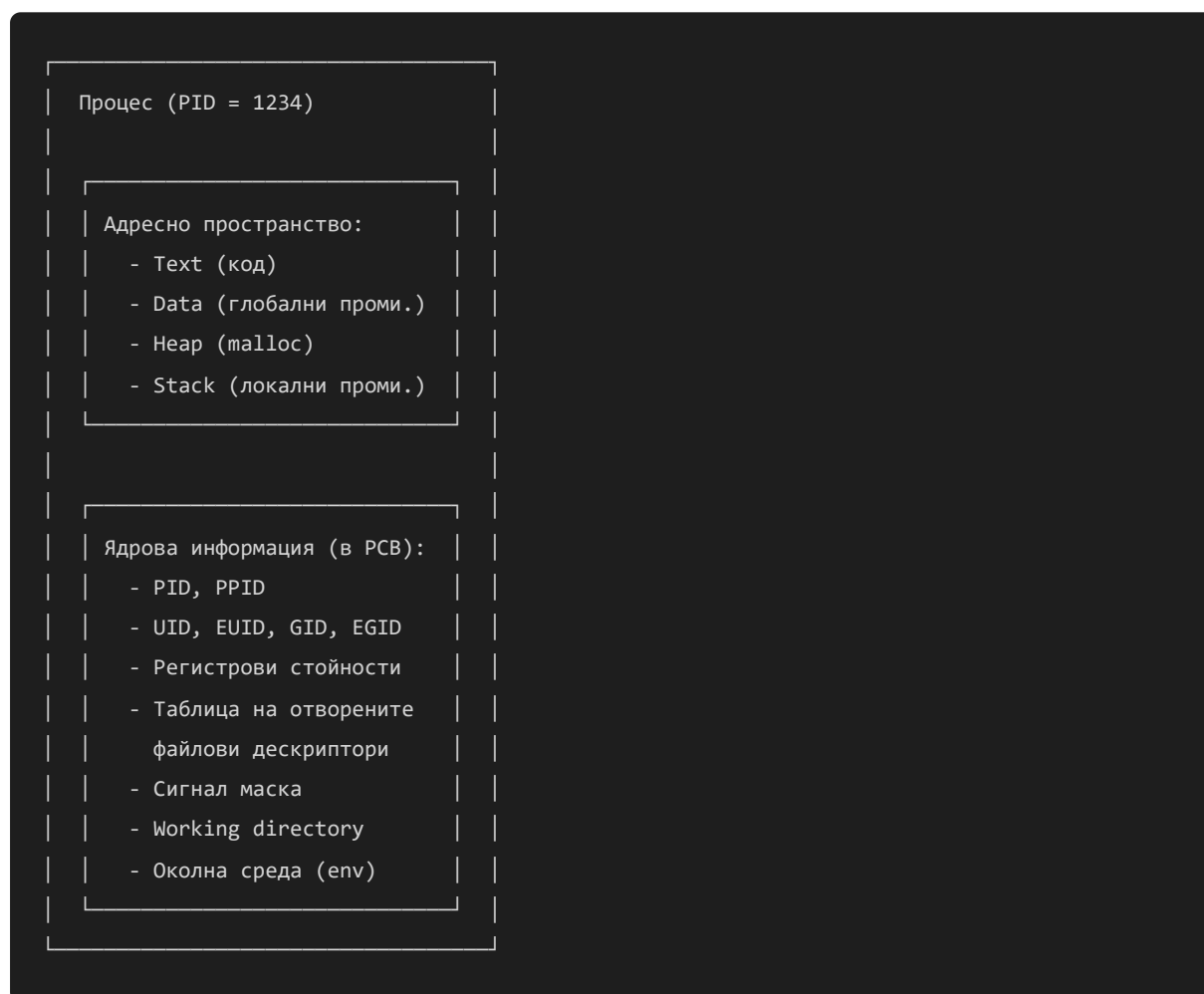
Системни примитиви за управление на процеси и междупроцесни комуникации

1. Какво е процес

Процес е изпълняваща се инстанция на програма. Програмата е статичен файл на диска (например `/bin/ls`), а процесът е тази програма заредена в паметта и в момента изпълнявана от CPU.

Менталната картина

Представи си, че процесът е "кутия", която съдържа:



Защо това е важно: Когато се прави `fork()`, всичко това се копира. Когато се прави `exec()`, **адресното пространство се заменя**, но ядровата информация (PID, отворените файлове, UID и др.) **се запазва**. Това е ключът към разбирането на цялата система.

Идентификатори

- **PID** (Process ID) – уникален номер на процеса.
- **PPID** (Parent PID) – PID на родителя.
- В Unix всеки процес има родител – дървовидна структура. Корен е `init` (PID 1) или `systemd`.

2. Основни системни примитиви

В Unix/POSIX има малък набор от примитиви, които заедно покриват целия жизнен цикъл на процеса:

Примитив	Какво прави	Кой го извиква
<code>fork()</code>	Създава нов процес (копие на текущия)	Родителят
<code>exec*()</code>	Заменя текущата програма с друга	Самият процес
<code>exit()</code>	Завършва процеса	Самият процес
<code>wait()</code> / <code>waitpid()</code>	Чака син да завърши	Родителят
<code>kill()</code>	Изпраща сигнал	Който и да е (с права)
<code>getpid()</code> / <code>getppid()</code>	Връща PID/PPID	Самият процес

Защо точно тези?

Това е **минималният набор**, който е необходим, за да:

- Стартираш нова програма (`fork` + `exec`)
- Дочакаш я (`wait`)
- Прекратиш я нормално или принудително (`exit`, `kill`)

Цялата идея на Unix shell-а се състои от тези примитиви.

3. Създаване на процес – `fork()`

Сигнатура

```
#include <unistd.h>
pid_t fork(void);
```

Какво прави

`fork()` създава **точно копие** на текущия процес. След извикването съществуват **два процеса**, които изпълняват **един и същ код**, започвайки от мястото веднага след `fork()`.

Странното – едно извикване, две връщания

Това е едно от най-объркващите неща за начинаещи. `fork()` се извиква **веднъж**, но връща **два пъти** – веднъж в родителя и веднъж в сина:

- В родителя `fork()` връща **PID на сина** (положително число).
- В сина `fork()` връща **0**.
- При грешка връща **-1** в родителя (синът не се създава).

Защо така?

Това е елегантен начин процесът да разпознае дали е родител или син, без да има нужда от допълнителен механизъм. По PID на сина родителят може после да го чака, да му праща сигнали и т.н.

Пример

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    } else if (pid == 0) {
        // Този код се изпълнява в СИНА
        printf("Аз съм синът. Моят PID е %d, родителят е %d\n",
               getpid(), getppid());
    } else {
        // Този код се изпълнява в РОДИТЕЛЯ
        printf("Аз съм родителят. Моят PID е %d, синът е %d\n",
               getpid(), pid);
        wait(NULL); // изчакваме сина
    }
    return 0;
}
```

Какво се наследява от родителя?

След `fork()` синът има свое **копие** на:

- Адресно пространство (код, данни, heap, stack)
- Отворени файлови дескриптори (същите inode-и, **споделят** file position)
- Сигнал маска и обработчици на сигнали
- Текуща директория
- UID, GID, EUID, EGID
- Околна среда (env)

Какво **НЕ** се наследява:

- PID, PPID (синът има нов PID, неговият PPID = PID на родителя)
- Стойности на чакащи сигнали (синът няма чакащи)

- Файлови ключалки (file locks)
- Таймери (alarm, interval timers)
- CPU време – нулира се

Copy-on-Write (COW) – защо `fork()` е бърз

На пръв поглед `fork()` изглежда скъп: копира цялото адресно пространство. Но модерните ОС използват **Copy-on-Write**:

1. След `fork()`, родителят и синът **споделят** едни и същи физически страници памет.
2. Страниците се маркират "read-only".
3. При опит за писане – процесорът прави page fault.
4. Ядрото копира **само тази страница**, дава я на писателя и я прави read-write.

Така че, ако след `fork()` синът веднага извика `exec()` (което заменя цялото адресно пространство), на практика нищо не е копирано. Това прави `fork()+exec()` много по-евтино, отколкото изглежда.

Интуиция

`fork()` е като "клонирание". Представи си, че програмата ти спира за момент, **точно същата програма в точно същото състояние** се клонира в нов процес, и след това и двете продължават да изпълняват кода. Единственото, по което се различават, е стойността, която `fork()` им е върнал.

Особени гранични случаи

- **Fork bomb**: `while(1) fork();` – експоненциално създава процеси и заразява системата. Защита: `ulimit`.
- **Зомби сина**: ако родителят не извика `wait()`, синът остава като "зомби" (вижте т.5–6).

4. Изпълнение на програма – `exec*()`

Защо ни трябва `exec` ?

`fork()` сам по себе си само дублира процеса. Но обикновено искаме синът да изпълнява **друга** програма – например когато shell-ът пуска `ls`, той прави:

```
shell → fork() → синът извиква exec("/bin/ls")
```

`exec* ()` **заменя** текущия процес (адресно пространство, код, данни) с нова програма. Същият PID, същият PPID, но **друга** програма се изпълнява.

Семейство функции `exec* ()`

Всички правят едно и също (заменят програмата), но се различават по начина, по който подаваш аргументи:

```
#include <unistd.h>

int execl(const char *path, const char *arg0, ..., NULL);
int execlp(const char *file, const char *arg0, ..., NULL);
int execlxe(const char *path, const char *arg0, ..., NULL, char *const envp[]);

int execv(const char *path, char *const argv[]);
int execvp(const char *file, char *const argv[]);
int execve(const char *path, char *const argv[], char *const envp[]);
```

Кодирай си наставките – мнемонично правило:

- **l** = **list** (списък аргументи разделени със запетая, завършващ с `NULL`)
- **v** = **vector** (масив `argv[]`, завършващ с `NULL`)
- **p** = **path** (търси изпълнимия файл в `PATH`)
- **e** = **environment** (даваш своя `envp[]`)

`execve ()` е **системното извикване** – всички останали са обвивки в `libc`, които накрая го извикват.

Пример

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        // В сина - изпълняваме ls -l /tmp
        execlp("ls", "ls", "-l", "/tmp", NULL);
        // Стигаме тук САМО при грешка - ехес върна -1
        perror("execlp failed");
        _exit(1);
    } else {
        wait(NULL);
    }
    return 0;
}
```

Какво се запазва, какво се сменя при `ехес`?

Сменя се (нова програма → ново всичко):

- Текстов сегмент (код)
- Данни (data, bss, heap)
- Stack
- Сигнал обработчиците (връщат се на default; маската се запазва)

Запазва се (същият процес):

- PID, PPID
- Отворените файлови дескриптори (освен ако са маркирани с `FD_CLOEXEC`)
- UID, GID, EUID, EGID (с изключение при setuid файлове – вж. т.7)
- Текуща директория
- Сигнал маската
- Околната среда (env) – освен ако подадеш нова

Интуиция

Мисли си за `exec` като "трансплантация на мозък". Тялото (PID, отворени файлове, родителски връзки) остава, но "мисленето" (кодът, данните) е напълно ново.

Защо `fork()` + `exec()`, а не комбиниран примитив?

Историческо решение на Unix, което се оказва **много мощно**. Между `fork()` и `exec()` синът може да:

- Затвори/отвори/пренасочи файлове (стандартен механизъм за shell pipelines: `cmd1 | cmd2`)
- Промени UID/GID (за demoting на права)
- Промени текущата директория
- Промени сигнал маската

Всичко това се прави в сина, преди да се замени с новата програма. Това е сърцето на shell-овете.

5. Завършване на процес – `exit()`

Три начина за нормално завършване

```
#include <stdlib.h>
void exit(int status);

#include <unistd.h>
void _exit(int status);

// или просто:
return n; // от main(), еквивалентно на exit(n)
```

Какво прави `exit(n)` ?

1. Извиква **функциите, регистрирани с** `atexit()`, в обратен ред на регистрацията.
2. Изхвърля (flush) и затваря всички stdio буфери (`stdout`, `stderr` и т.н.).

3. Изтрива временните файлове, създадени с `tmpfile()`.
4. Извиква `_exit(n)` накрая.

Какво прави `_exit(n)` ?

Това е системното извикване. То:

1. Затваря всички отворени файлови дескриптори.
2. Освобождава цялата памет на процеса.
3. Прави всички "сирачета" (orphans) на сина (ако имаше деца, те минават под `init`).
4. Изпраща сигнал `SIGCHLD` на родителя.
5. Превръща процеса в **зомби** – запазва се само минимум информация (PID, статус, ресурси), докато родителят не извика `wait()`.

Разлика между `exit()` и `_exit()`

	<code>exit()</code>	<code>_exit()</code>
atexit handlers	Да	Не
stdio flush	Да	Не
Системно извикване	Не (libc обвивка)	Да
Кога да го ползваме	Нормално прекратяване	След <code>fork()</code> в син, когато <code>exec()</code> пропадне

Защо след неуспешен `exec()` използваме `_exit()` ? Защото синът все още споделя stdio буферите с родителя (преди flush). Ако извика `exit()`, ще flush-не буферите и съдържанието ще се изведе **два пъти** (веднъж от сина, веднъж от родителя).

Регистриране на функции с `atexit()`

```
#include <stdlib.h>

void cleanup1(void) { puts("cleanup 1"); }
void cleanup2(void) { puts("cleanup 2"); }

int main(void) {
    atexit(cleanup1);
    atexit(cleanup2);
    // изход в обратен ред: cleanup2, после cleanup1
    return 0;
}
```

Завършване чрез сигнал (ненормално)

Процесът може да бъде убит със сигнал (`SIGKILL`, `SIGSEGV`, `SIGTERM` и т.н.). Тогава статусът на процеса няма да съдържа `exit` код, а номер на сигнала, който го е убил. Това се различава с макроси `WIFEXITED` vs `WIFSIGNALED` (вж. т.6).

Зомби и сираци

- **Зомби (zombie):** процес, който е завършил, но родителят още не е извикал `wait()`. Заема място в process table-a. Виждаш ги в `ps` като `Z`.
- **Сирак (orphan):** процес, чийто родител е умрял преди него. Веднага се "осиновява" от `init` (PID 1), който периодично извиква `wait()` и така чисти зомбита.

Затова за дълготрайни демони използваме техниката "double fork":

```
parent → fork → child → fork → grandchild
      child exits immediately
      ↓
      grandchild е осиновен от init,
      и init ще го wait()-не,
      така че grandchild никога не става зомби.
```

6. Синхронизация със завършването на процеса-син

Защо родителят чака?

1. Да получи **изходния статус** на сина – дали е завършил успешно.
2. Да позволи на ОС да освободи ресурсите на сина (за да не остане зомби).
3. Да синхронизира работата си с края на сина (например shell чака команда да приключи).

`wait()`

```
#include <sys/wait.h>
pid_t wait(int *status);
```

- Блокира, докато **някое** дете не завърши.
- Връща PID на завършилото дете.
- В `*status` записва информация за края.
- Връща `-1` ако няма деца.

`waitpid()`

```
pid_t waitpid(pid_t pid, int *status, int options);
```

По-гъвкав вариант:

- `pid > 0` – чака конкретно дете.
- `pid == 0` – чака което и да е дете от същата process group.
- `pid == -1` – еквивалентно на `wait()` (което и да е дете).
- `pid < -1` – чака дете от process group `|pid|`.

Опции:

- `WNOHANG` – не блокирай. Ако няма завършило дете, върни 0.
- `WUNTRACED` – върни и за деца, които са спрени (от `SIGSTOP`).
- `WCONTINUED` – върни и за деца, които са продължили (от `SIGCONT`).

Декодиране на статуса

`status` е цяло число, в което различни битове кодират различни неща. Никога не го интерпретирай директно – използвай макросите:

```
if (WIFEXITED(status)) {
    // детето е завършило нормално чрез exit()
    int code = WEXITSTATUS(status);    // exit код (0-255)
}
if (WIFSIGNALED(status)) {
    // детето е убито от сигнал
    int sig = WTERMSIG(status);        // номер на сигнала
    if (WCOREDUMP(status)) {
        // създаден е core dump
    }
}
if (WIFSTOPPED(status)) {
    // детето е спряно (debugger)
    int sig = WSTOPSIG(status);
}
if (WIFCONTINUED(status)) {
    // детето е продължило след SIGCONT
}
```

Пълен пример

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        // В сина
        sleep(1);
        return 42;
    }
    int status;
    pid_t w = waitpid(pid, &status, 0);
    printf("Синът %d завърши\n", w);

    if (WIFEXITED(status)) {
        printf("  exit код = %d\n", WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("  убит от сигнал %d\n", WTERMSIG(status));
    }
    return 0;
}
```

Асинхронно изчакване чрез SIGCHLD

Когато детето завърши, ядрото изпраща SIGCHLD на родителя. Можеш да закачиш обработчик, който вика waitpid() с WNOHANG :

```
void sigchld_handler(int sig) {
    while (waitpid(-1, NULL, WNOHANG) > 0)
        ; // прибираме всички зомбита
}

// в main:
signal(SIGCHLD, sigchld_handler);
```

Така родителят не трябва да блокира – върши си работата, а когато дете завърши, обработчикът се извиква.

Внимание: При множество едновременно завършващи деца, сигнали могат да се "сливат" (не се опашкуват). Затова в обработчика се вика `waitpid` в `while` - цикъл с `WNOHANG`, за да приберем **всички** готови деца.

Интуиция

Мисли си за `wait` като "разписка" – синът оставя exit статуса си в ядрото, а родителят минава да го "вземе". Докато не вземе разписката, синът остава като зомби (затворил магазина, но още не е предал отчета).

7. Права на процеси – потребителски идентификатори

UID и GID – какво и защо

Всеки процес има няколко идентификатора, които определят **с какви права** изпълнява операции:

- **UID** (User ID) – потребител, който притежава процеса.
- **GID** (Group ID) – основна група.
- На една машина могат да съществуват хиляди UID-и; UID 0 е `root` (суперпотребител).

В Unix процесът има **три варианта** на UID:

Идент.	Име	За какво служи
<code>ruid</code>	Real UID	Кой реално стартира процеса
<code>euid</code>	Effective UID	Кой "се прави, че е" процесът за проверка на права
<code>suid</code>	Saved UID	Запазена версия, за да може да се връща

Същото има и за GID: `rgid`, `egid`, `sgid`.

Защо има нужда от три различни?

Причина: механизмът `setuid` на изпълними файлове.

Класически пример: `/usr/bin/passwd`. Този файл сменя паролата на потребителя, която се пази в `/etc/shadow` – файл, който е достъпен **само** за root. Но обикновен потребител трябва да може да си смени паролата!

Решение: файлът `passwd` е маркиран с **setuid бит** (`chmod u+s`). Когато потребител го изпълни:

- `ruid` = неговият UID (например 1000)
- `euid` = UID на собственика на файла (root, 0)

Така проверките за права се правят като root (защото `euid = 0`), но процесът знае **кой реално** го стартира (`ruid = 1000`).

Сценарий: passwd проверява права

```
ruid = 1000    (Иван)
euid = 0      (root, заради setuid бита)
suid = 0      (запазено)
```

`passwd` чете и пише в `/etc/shadow` (като root). Но добре написана setuid програма може **временно да върне** `euid` обратно на 1000 (с `seteuid(1000)`), за да направи нещо като обикновен потребител, и после пак да си вземе rights чрез `seteuid(0)` – защото `suid = 0` пази "правото" да се върне.

Функции

```
#include <unistd.h>

uid_t getuid(void);      // real UID
uid_t geteuid(void);     // effective UID
gid_t getgid(void);
gid_t getegid(void);

int setuid(uid_t uid);   // променя ruid и euid (и suid за root)
int seteuid(uid_t uid);  // променя само euid (към ruid, suid, или 0 за root)
int setreuid(uid_t r, uid_t e);
int setresuid(uid_t r, uid_t e, uid_t s); // Linux, най-фин контрол
```

Правила:

- Само `root` може да сменя UID на каквото иска.
- Обикновен потребител може само да си върне `euid` към `ruid` или `suid`.

Setuid бит

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 88496 ... /usr/bin/passwd
^
s вместо x - setuid бит
```

`s` в позицията на `x` за собственика означава setuid. Когато изпълнимият файл се стартира:

- Без setuid: `euid` = `ruid` = UID на потребителя.
- Със setuid: `ruid` = UID на потребителя, `euid` = UID на собственика на файла.

Setgid битът работи аналогично за GID-а.

Защо `euid` се ползва, а не `ruid`?

`euid` определя **какви права има процесът точно в момента**, защото:

- `open()`, `unlink()`, `chmod()` и др. проверяват `uid` (и `egid`).
- Сигнали (`kill`) обикновено могат да се пращат, ако `ruid` или `uid` на изпращача съвпадат с `ruid` или `suid` на получателя.

Интуиция

- `uid` = "кой съм наистина" (потребителят, който стартира).
- `uid` = "какъв се правя в момента" (за проверка на права).
- `suid` = "какъв мога да стана пак, ако се променя" (запазено състояние).

Това е като служебна карта: реално си Иван (`uid`), но имаш достъп до `root`-стая с временна карта (`uid=root`), а оригиналната карта на `root` е заключена в чекмеджето (`suid=root`), за да си я вземеш пак, ако я върнеш.

Сигурност

`setuid root` програми са много опасни и често източник на CVE-та (буфер овърфлоу, повишаване на права). Затова:

- Минимизирай времето, в което си `uid=0`.
- Не пускай дъщерни процеси с `exec` като `root`, ако не трябва.
- Linux предлага `capabilities` – по-фин контрол вместо all-or-nothing `setuid`.

8. Междупроцесни комуникации – преглед

Защо ни трябва?

Един процес е изолиран – има свое адресно пространство. Това е добре за стабилност (един срив не убива другите) и сигурност (един процес не вижда данните на друг), но **процеси често трябва да си говорят**:

- Shell pipeline: `ls | grep foo | wc -l` – три процеса си предават данни.
- База данни: десетки процеси/нишки обработват клиенти и споделят буфери.
- GUI: процес на приложението говори с X сървър.

Класификация на IPC механизмите

Механизъм	Тип	Скорост	Сложност
Сигнали	Уведомление (1 бит инфо)	Бърза	Ниска
Pipe ()	Поток от байтове, родствени	Средна	Ниска
FIFO (named pipe)	Поток, всякакви процеси	Средна	Ниска
System V IPC	Стара система (msg, sem, shm)	Различна	Висока
POSIX shared memory	Споделена памет	Най-бърза	Средна
POSIX semaphores	Синхронизация	Бърза	Средна
POSIX message queues	Съобщения с приоритети	Средна	Средна
Sockets	Мрежа или локално	Средна	Средна

В следващите секции разглеждаме сигналите, каналите и POSIX IPC.

Основни въпроси, които всеки IPC механизъм решава

1. **Именуване** – как процес А намира канал, който процес В е създал?
2. **Синхронизация** – какво се случва ако четецът чете, преди писателят да е писал?
3. **Капацитет** – колко данни могат да чакат в буфера?
4. **Семантика на множествени читатели/писатели** – race conditions?
5. **Живот** – какво става с канала, когато процесите умрат?

9. Сигнали

Какво е сигнал?

Сигналът е **софтуерно прекъсване** – уведомление, че се е случило събитие. Носи минимална информация – само номера си (и евентуално допълнителна структура `siginfo_t` при real-time signals).

Когато процесът получи сигнал, текущото му изпълнение се **прекъсва** и се изпълнява **обработчик** (signal handler) – функция, която обикновено е предварително регистрирана.

Кога се пращат сигнали?

1. **Хардуерни събития:** деление на нула (`SIGFPE`), невалидно обръщение към памет (`SIGSEGV`), невалидна инструкция (`SIGILL`).
2. **Терминал:** `Ctrl+C` → `SIGINT` , `Ctrl+\` → `SIGQUIT` , `Ctrl+Z` → `SIGTSTP` .
3. **Друг процес чрез** `kill()` .
4. **Ядрото:** `SIGCHLD` при завършване на дете, `SIGPIPE` при писане в затворен pipe, `SIGALRM` при изтичане на таймер.
5. **Самият процес:** `raise(SIGABRT)` , `abort()` .

Основни сигнали

Сигнал	Номер (приблизително)	Default действие	Може ли да се прихване
<code>SIGHUP</code>	1	Terminate	Да
<code>SIGINT</code>	2	Terminate	Да
<code>SIGQUIT</code>	3	Terminate + core	Да
<code>SIGILL</code>	4	Terminate + core	Да
<code>SIGABRT</code>	6	Terminate + core	Да
<code>SIGFPE</code>	8	Terminate + core	Да
<code>SIGKILL</code>	9	Terminate	НЕ
<code>SIGSEGV</code>	11	Terminate + core	Да
<code>SIGPIPE</code>	13	Terminate	Да
<code>SIGALRM</code>	14	Terminate	Да
<code>SIGTERM</code>	15	Terminate	Да
<code>SIGUSR1</code>	10	Terminate	Да
<code>SIGUSR2</code>	12	Terminate	Да
<code>SIGCHLD</code>	17	Ignore	Да
<code>SIGSTOP</code>	19	Stop	НЕ
<code>SIGCONT</code>	18	Continue	Да

`SIGKILL` и `SIGSTOP` не могат да се прихванат, игнорират или блокират – това е по-дизайн, за да има винаги начин операторът да убие процес.

Default действия

- **Terminate** – процесът умира.
- **Terminate + core** – умира и сваля core dump (за дебъгване).
- **Ignore** – нищо не се случва.
- **Stop** – процесът се спира (като `Ctrl+Z`).
- **Continue** – продължава след стоп.

Регистриране на обработчик

Стар API: `signal()`

```
#include <signal.h>
void (*signal(int signum, void (*handler)(int)))(int);
```

```
void handler(int sig) {
    printf("Получих сигнал %d\n", sig);
}

signal(SIGINT, handler);
```

Проблем: Поведението на `signal()` е различно между Unix вариантите (BSD vs System V). По-добре използвай `sigaction()`.

Модерен API: `sigaction()`

```
#include <signal.h>

struct sigaction {
    void      (*sa_handler) (int);           // прост обработчик
    void      (*sa_sigaction) (int, siginfo_t *, void *); // подробен
    sigset_t   sa_mask;                     // сигнали, блокирани докато тече об
    p.
    int        sa_flags;                     // флагове
};

int sigaction(int signum, const struct sigaction *act,
              struct sigaction *oldact);
```

Пример:

```
void handler(int sig) {
    write(STDOUT_FILENO, "Hit!\n", 5);
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART; // ако сигналът прекъсне системно извикване,
                             // да се рестартира автоматично

    sigaction(SIGINT, &sa, NULL);

    while (1) pause(); // блокирай, докато не дойде сигнал
}
```

Изпращане на сигнал – `kill()`

```
#include <signal.h>
int kill(pid_t pid, int sig);
int raise(int sig);    // на самия себе си
```

- `pid > 0` – на конкретен процес.
- `pid == 0` – на всички в process group-a.
- `pid == -1` – на всички процеси (с подходящи права).
- `pid < -1` – на process group `|pid|`.
- `sig == 0` – не праща сигнал, само проверява дали процесът съществува и има ли права.

Сигнал маски и блокиране

Можеш временно да **блокираш** сигнали (не да ги пропуснеш – те ще чакат опашка):

```
sigset_t mask;
sigemptyset(&mask);
sigaddset(&mask, SIGINT);
sigprocmask(SIG_BLOCK, &mask, NULL);    // блокирай SIGINT
// ... critical section ...
sigprocmask(SIG_UNBLOCK, &mask, NULL);    // разблокирай – чакащият SIGINT сега ще
дойде
```

Защо? Защото обикновените сигнали **не се опашкуват** – ако два пъти дойде SIGINT, докато е блокиран, ще се обработи **веднъж**. (Real-time сигналите – `SIGRTMIN..SIGRTMAX` – се опашкуват.)

Async-signal-safe функции

В обработчика на сигнал **не можеш да правиш каквото си искаш**. Защото сигналът прекъсва процеса в **произволно** място – може да си в средата на `malloc()`, например. Ако обработчикът пак извика `malloc()`, ще има deadlock или corruption.

POSIX дефинира списък на **async-signal-safe** функции, които могат да се ползват в обработчик. Те включват: `write`, `read`, `_exit`, `signal`, `sigaction`, `kill`, `getpid` и др. **НЕ включват:** `printf`, `malloc`, `free`, `exit`.

Затова, ако трябва да направиш нещо в обработчика, най-добре:

```
volatile sig_atomic_t flag = 0;

void handler(int sig) {
    flag = 1;    // само присвояване на флаг
}

// main loop:
while (1) {
    if (flag) {
        // обработи събитието безопасно тук
        flag = 0;
    }
    // ... останала работа ...
}
```

`alarm()` И `pause()`

```
unsigned int alarm(unsigned int seconds);    // SIGALRM след n секунди
int pause(void);                            // блокирай до следващ сигнал
```

Класически "timeout" механизъм:

```
signal(SIGALRM, alarm_handler);
alarm(5);                // задай 5 секунди
result = read(fd, buf, n); // блокиращо четене
alarm(0);                // отмени, ако е приключило по-рано
```

Ако след 5 секунди няма данни, обработчикът се извиква, `read` се прекъсва (връща -1 с `errno=EINTR`).

Интуиция

- Сигналът е като **тропане на вратата** – чуваш че се е случило нещо, но не получаваш съдържание.
- Прекъсва те **точно сега** – няма обещание кога точно.
- Може да изпуснеш повтарящи се чукания – не разчитай на брояч.

10. Програмни канали (pipes)

Какво е pipe?

Pipe е едностранен (unidirectional) канал – байтов поток. Има **четяща** страна и **пищеща** страна. Каквото запишеш в пишещата, можеш да прочетеш от четящата (FIFO ред).

Неименуван (unnamed) pipe – `pipe()`

```
#include <unistd.h>
int pipe(int fd[2]);
```

Връща два файлови дескриптора:

- `fd[0]` – четене (read end)
- `fd[1]` – писане (write end)

Защо работи само между родствени процеси?

`pipe()` живее в ядрото. Двата `fd` са валидни **само в процеса**, който е извикал `pipe()`. За да може друг процес да го ползва, трябва да наследи дескрипторите чрез `fork()`. Затова `pipe()` работи между родител и син (или между братя, ако родителят им е дал дескрипторите).

Стандартна употреба: shell pipeline

cmd1 | cmd2

```
int fd[2];
pipe(fd);

if (fork() == 0) {
    // дете 1 - cmd1
    close(fd[0]);           // не четем
    dup2(fd[1], STDOUT_FILENO); // stdout → pipe
    close(fd[1]);
    execlp("cmd1", "cmd1", NULL);
}
if (fork() == 0) {
    // дете 2 - cmd2
    close(fd[1]);           // не пишем
    dup2(fd[0], STDIN_FILENO); // stdin ← pipe
    close(fd[0]);
    execlp("cmd2", "cmd2", NULL);
}
close(fd[0]);
close(fd[1]);
wait(NULL); wait(NULL);
```

Защо `close()` на ненужните краища? Защото докато **поне един** процес държи пишещата страна отворена, четецът няма да получи EOF. Ако забравиш да затвориш, ще има deadlock.

Поведение на четене/писане

Ситуация	Поведение
Четене от pipe с данни	Връща данни
Четене от празен pipe, но с отворен writer	Блокира
Четене от празен pipe, всички writers затворени	Връща 0 (EOF)
Писане в pipe с място	Записва
Писане в пълен pipe (буферът е ~64 KB)	Блокира
Писане в pipe без читатели	<code>SIGPIPE</code> (по подразбиране убива процеса)

Атомарност

POSIX гарантира: писане на до `PIPE_BUF` байта (обикновено 4096 или 512) е **атомарно** – няма да се смеси с друго писане. По-големи блокове могат да се препокрият.

Именуван pipe – FIFO (`mkfifo`)

Решава проблема, че `pipe()` работи само между родствени процеси.

```
#include <sys/types.h>
#include <sys/stat.h>
int mkfifo(const char *pathname, mode_t mode);
```

FIFO е специален файл в файловата система. Два съвсем независими процеса могат да го отворят:

```
// процес 1 (писател)
int fd = open("/tmp/myfifo", O_WRONLY);
write(fd, "hello", 5);

// процес 2 (четец)
int fd = open("/tmp/myfifo", O_RDONLY);
read(fd, buf, n);
```

Семантика: `open()` блокира, докато другата страна също не отвори. (Освен ако ползваш `O_NONBLOCK`.)

FIFO живее, докато не го изтриеш с `unlink()` или `rm`. Но докато няма процеси, които го държат отворен, данните в него **не се пазят между сесии**.

Интуиция

- `pipe()` = "вътрешна тръба" между родителя и сина.
- `mkfifo` = "обществена тръба" – всеки, който знае пътя ѝ, може да я ползва.
- В двата случая е **байтов поток** – няма граници на съобщенията. Ако пишеш `write(fd, "ABC", 3)` и `write(fd, "DEF", 3)`, четецът може да прочете `"ABCDEF"` наведнъж или `"AB"` и после `"CDEF"`.

11. POSIX IPC – Споделена памет

Защо споделена памет?

Всички предишни IPC механизми **копират данни** между процесите (през ядрото). За голяма обмяна на данни това е бавно: ядрото копира байтовете от user space на писателя в kernel буфер, после в user space на четеща. Това са две копирания.

Споделената памет елиминира това: ядрото маркира една и съща физическа памет като част от адресните пространства на двата процеса. След това четенето и писането става **директно**, без системни извиквания.

Цена: **синхронизацията е твоя отговорност** – няма автоматично "изчакай, докато има данни". Затова споделената памет почти винаги се комбинира със семафори или мютекси.

API: `shm_open` + `mmap`

Три стъпки за всеки процес:

1. **Отвори** обект на споделена памет: `shm_open()`.
2. **Размер**: само създателят – `ftruncate()`.
3. **Мар** в адресното пространство: `mmap()`.

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int shm_open(const char *name, int oflag, mode_t mode);
int shm_unlink(const char *name);

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
int munmap(void *addr, size_t length);
```

Пример: писател

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define NAME "/myshm"
#define SIZE 4096

int main(void) {
    int fd = shm_open(NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, SIZE);

    char *p = mmap(NULL, SIZE, PROT_READ | PROT_WRITE,
                    MAP_SHARED, fd, 0);
    strcpy(p, "Hello from writer!");
    munmap(p, SIZE);
    close(fd);
    return 0;
}
```

Пример: четец

```
int fd = shm_open(NAME, O_RDONLY, 0);
char *p = mmap(NULL, SIZE, PROT_READ, MAP_SHARED, fd, 0);
printf("Прочетено: %s\n", p);
munmap(p, SIZE);
close(fd);
shm_unlink(NAME); // премахни обекта
```

Какво прави `mmap()` ?

Превръща файлов дескриптор в указател към памет. След `mmap()` :

- Достъпът до `p[i]` е като достъп до обикновен масив.

- Промените се виждат от **ВСИЧКИ** процеси, които са направили `mmap()` на същия обект с `MAP_SHARED`.

Имена

POSIX shared memory обектите имат имена, започващи с `/` (по конвенция). Те живеят в `/dev/shm/` (на Linux).

```
$ ls /dev/shm/  
myshm
```

`shm_unlink`

Премахва обекта от namespace. Но докато има процеси, които са го отворили, паметта живее – чак след последния `close()` се освобождава.

Защо имаме нужда от синхронизация

```
// процес 1  
counter++;  
  
// процес 2 (по същото време)  
counter++;
```

Резултат: counter може да стане 1 (а не 2), защото и двата прочетоха старата стойност преди да я променят. Това е **race condition**.

Решение: семафор или мютекс. Виж следващата секция.

Структура с мютекс в споделена памет (Linux)

```
typedef struct {
    pthread_mutex_t mutex;
    int counter;
} shared_data_t;

// при създаване (писател):
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setshared(&attr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&data->mutex, &attr);
```

Важно: трябва `PTHREAD_PROCESS_SHARED`, иначе мютексът работи само между нишки на един процес.

Интуиция

- Споделената памет е като **обща бяла дъска** – всеки може да пише, всеки може да чете, но никой не контролира кога.
- Затова трябва "правило за дъската" – например "който държи маркера може да пише" – това е семафорът.

12. POSIX IPC – Семафори

Какво е семафор?

Семафорът е **целочислен брояч с атомарни операции**:

- `wait` (или `P`, `acquire`, `down`): ако брояч $> 0 \rightarrow$ намали с 1 и продължи; ако брояч $= 0 \rightarrow$ блокирай.
- `post` (или `V`, `release`, `up`): увеличи брояча с 1; ако някой чака, събуди го.

Това е елементарен примитив за синхронизация.

Видове семафори

1. **Counting** – броячът може да бъде >1 , например ресурси (5 свободни принтера).
2. **Binary** – броячът е 0 или 1, работи като мютекс (взаимно изключване).

POSIX семафори: именувани и неименувани

Именуван – има име в namespace, споделя се между несвързани процеси:

```
#include <semaphore.h>
sem_t *sem_open(const char *name, int oflag, mode_t mode, unsigned int value);
int sem_close(sem_t *sem);
int sem_unlink(const char *name);
```

Неименуван – живее в памет, споделя се чрез споделена памет (или между нишки):

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
int sem_destroy(sem_t *sem);
```

Основни операции

```
int sem_wait(sem_t *sem);           // блокиращо
int sem_trywait(sem_t *sem);        // неблокиращо (върща -1 с EAGAIN)
int sem_timedwait(sem_t *sem, const struct timespec *abs_timeout);
int sem_post(sem_t *sem);
int sem_getvalue(sem_t *sem, int *sval);
```


Класически пример: producer-consumer

```
sem_t *empty; // брой свободни слотове в буфера
sem_t *full;  // брой пълни слотове
sem_t *mutex; // достъп до буфера

empty = sem_open("/empty", O_CREAT, 0666, N); // N свободни в началото
full  = sem_open("/full",  O_CREAT, 0666, 0); // 0 пълни в началото
mutex = sem_open("/mutex", O_CREAT, 0666, 1); // 1 за mutex

// PRODUCER
while (1) {
    item = produce();
    sem_wait(empty); // изчакай свободен слот
    sem_wait(mutex); // вземи ключа
    buffer_put(item);
    sem_post(mutex); // върни ключа
    sem_post(full);  // съобщи - има още един пълен
}

// CONSUMER
while (1) {
    sem_wait(full); // изчакай пълен слот
    sem_wait(mutex);
    item = buffer_get();
    sem_post(mutex);
    sem_post(empty); // съобщи - има още един свободен
    consume(item);
}
```

Защо точно тази последователност? Ако сложиш `sem_wait(mutex)` преди

`sem_wait(empty)`, ето какво се случва:

1. Producer взема mutex (буферът е пълен, няма empty).
2. Producer чака на `sem_wait(empty)` – но държи mutex.
3. Consumer иска mutex (за да освободи слот), блокира.
4. **Deadlock.**

Затова **първо** изчакваш ресурс, **после** взимаш mutex.

Семафор vs мютекс

	Семафор	Мютекс
Има концепция за "собственик"	Не	Да (този, който е взел, го пуска)
Може ли да бъде "пуснат" от друг	Да	Не (поведение зависи от API)
Брой	Произволно цяло	0 или 1
Употреба	Брояч на ресурси, сигнализиране	Защита на критична секция

Интуиция

- Семафорът е като **жетони на пералнята**: имаш N жетона, всеки иска един, ако няма – чакаш. Като изпере, връща жетон.
- Бинарният семафор като 1 жетон е "ключ за стаята" – само един може да влезе.

13. POSIX IPC – Съобщителни опашки

Какво е message queue?

Опашка от **дискретни съобщения**. За разлика от pipe-овете (които са поток от байтове без граници), всяко съобщение тук е отделен пакет с фиксиран максимален размер.

Особености:

- Приоритети** – съобщенията могат да имат приоритет; високите се получават първи.
- Граници на съобщенията** – едно `mq_receive()` връща точно едно съобщение.
- Постоянство** – опашката живее, докато не я `unlink`-неш или рестартираш системата.

API

```
#include <mqueue.h>

mqd_t mq_open(const char *name, int oflag, mode_t mode,
              struct mq_attr *attr);
int mq_close(mqd_t mqdes);
int mq_unlink(const char *name);

int mq_send(mqd_t mqdes, const char *msg_ptr, size_t msg_len,
            unsigned int msg_prio);
ssize_t mq_receive(mqd_t mqdes, char *msg_ptr, size_t msg_len,
                  unsigned int *msg_prio);

int mq_getattr(mqd_t mqdes, struct mq_attr *attr);
int mq_setattr(mqd_t mqdes, const struct mq_attr *new, struct mq_attr *old);

int mq_notify(mqd_t mqdes, const struct sigevent *notification);
```

Атрибути

```
struct mq_attr {
    long mq_flags;      // 0 или O_NONBLOCK
    long mq_maxmsg;     // максимален брой съобщения в опашката
    long mq_msgsize;    // максимален размер на едно съобщение
    long mq_curmsgs;    // текущ брой съобщения (read-only)
};
```

Пример: писател

```
#include <mqueue.h>
#include <string.h>

int main(void) {
    struct mq_attr attr = {
        .mq_flags = 0,
        .mq_maxmsg = 10,
        .mq_msgsize = 256,
    };
    mqd_t mq = mq_open("/myqueue", O_CREAT | O_WRONLY, 0666, &attr);
    const char *msg = "Hello!";
    mq_send(mq, msg, strlen(msg) + 1, 5);    // приоритет 5
    mq_close(mq);
    return 0;
}
```

Пример: четец

```
mqd_t mq = mq_open("/myqueue", O_RDONLY);
char buf[256];
unsigned int prio;
ssize_t n = mq_receive(mq, buf, sizeof(buf), &prio);
printf("Получено (prio=%u): %s\n", prio, buf);
mq_close(mq);
mq_unlink("/myqueue");
```

Поведение

- `mq_send` на пълна опашка – блокира (или връща `EAGAIN`, ако `O_NONBLOCK`).
- `mq_receive` на празна опашка – блокира.
- `mq_receive` винаги връща съобщението с **най-висок приоритет** (при равни – FIFO).
- Размерът на буфера в `mq_receive` трябва да е **поне** `mq_msgsize`, иначе получаваш грешка `EMSGSIZE`.

Асинхронно уведомяване – `mq_notify`

Може да зададеш съобщение **да бъде уведомен** (чрез сигнал или нишка), когато в празна опашка пристигне ново съобщение. Полезно за event-driven сървъри.

Сравнение с pipe

Характеристика	Pipe/FIFO	Message Queue
Граници на съобщения	Не (byte stream)	Да
Приоритети	Не	Да
Постоянство	Само докато има отворили	Докато не <code>unlink</code>
Atomic write	До PIPE_BUF	Винаги по едно съобщение
Multi-producer	Има race	Atomic

Интуиция

- **Pipe** = непрекъсната тръба, в която шептиш – думите се сливат в поток.
- **Message Queue** = пощенска кутия – всяко писмо е отделно, с печат за приоритет.

Заключителен синтез – как се връзва всичко

Сценарий: shell изпълнява `ls -l | wc -l`

1. Shell-ът прави `pipe()` → получава `fd[0]`, `fd[1]`.
2. Прави `fork()` за `ls`:
 - В сина: `dup2(fd[1], 1)`, `close(fd[0])`, `close(fd[1])`, `execvp("ls", ...)`.
3. Прави `fork()` за `wc`:
 - В сина: `dup2(fd[0], 0)`, `close(fd[1])`, `close(fd[0])`, `execvp("wc", ...)`.
4. В родителя: `close(fd[0])`, `close(fd[1])`, `wait(NULL)`; `wait(NULL)`;
5. Ако натиснеш `Ctrl+C`: ядрото изпраща `SIGINT` на foreground групата – всички получават сигнала и завършват. Shell получава `SIGCHLD` от ядрото и прибира зомбитата.

В тази миниатюра виждаш как **fork + exec + pipes + signals + wait + UID контрол** работят заедно. Това е същността на Unix.

Кога кой механизъм да избира?

Нуждаете се от...	Изберете
Уведомление "случи се събитие"	Сигнал
Поточно прехвърляне на байтове между родствени процеси	Pipe
Поточно прехвърляне между независими процеси	FIFO или Unix socket
Дискретни съобщения с приоритети	Message queue
Максимална скорост, споделени структури	Shared memory + семафор
Само синхронизация (без данни)	Семафор
Защита на критична секция в един процес	Mutex
Мрежа или локално	Sockets

Финални съвети за подготовка за изпит

1. **Знай защо**, не само какво. Защо `fork()` връща два пъти? Защо има `euid` отделно от `uid`? Защо `SIGKILL` не може да се прихване?
2. **Помни проблемите**: zombie, orphan, deadlock в producer-consumer, EINTR при сигнал.
3. **Можеш да напишеш**: минимален shell с pipe; producer-consumer със семафори; обработчик на `SIGCHLD`.
4. **Не бъркай**: `exit()` vs `_exit()`; `signal()` vs `sigaction()`; pipe vs FIFO vs message queue; System V IPC vs POSIX IPC.

Допълнителни ресурси за дълбочина

- **APUE** (Advanced Programming in the UNIX Environment, W. Richard Stevens) – библията.
- **TLPI** (The Linux Programming Interface, Michael Kerrisk) – Linux-специфично, изключително изчерпателна.
- `man 2 fork`, `man 2 execve`, `man 7 signal`, `man 7 pipe`, `man 7 mq_overview`, `man 7 shm_overview`, `man 7 sem_overview`.